

SOFTWARE REQUIREMENTS SPECIFICATION

NexusChat

A Lightweight Chat Application with AI Summaries, Translation, and Search

Document Version	5.0
Author	VT
Date	August 2, 2026
Status	Draft for Review
Project Type	Student / Academic Project

A focused, achievable scope for a solo/academic build: two chat modes, three AI-assisted utilities, no bloat.

Table of Contents

1 Introduction

1.1 Purpose

This document defines the requirements for NexusChat, a lightweight web-based chat application built as a student/academic project. It is scoped to two core chat experiences, plus three focused AI-assisted utilities (summarization, translation, and search), so the project remains achievable for a solo or small-team build while still demonstrating real-time systems, database design, and Generative AI integration.

This document is written so any reader — a classmate, instructor, or evaluator — can understand exactly what the app does and does not do, without needing further explanation.

1.2 Scope

NexusChat provides exactly two chat modes, each supported by the same three AI utilities:

- Temporary Chatroom — a multi-user room that anyone can be invited to, supports text and media sharing, and is automatically (or manually) destroyed when the admin deletes it or all members go offline. Before it disappears, an AI-generated summary of the conversation is posted into the room as a system message for every member to see.
- Private Chat — a simple one-to-one conversation (WhatsApp-style) that persists indefinitely. Either user can manually delete the conversation, or both users can opt into a recurring 24-hour auto-reset that clears the chat automatically.

Both chat modes additionally support:

- AI Translation — any message can be translated into a user's preferred language on request.
- Message Search — users can search within a chatroom or a private chat by word or sentence to locate past messages.
- Message Actions — a sender can delete/unsend, edit (text only), copy, or forward their own messages.
- Reactions & Stickers — users can react to any message with an emoji, and send static stickers as their own message type.

Everything outside these seven capabilities — group admin hierarchies, friend/contact systems, message pinning, reply-to threading, smart replies, animated/custom stickers, voice/video calling, and similar additions — is explicitly out of scope for this version. This keeps the build small enough to finish and defend as a course project.

1.3 Objectives

Objective	Description
Keep the scope small and finishable	Two chat modes and three AI utilities — each simple enough to fully implement and test within a student timeline.
Demonstrate real-time messaging	WebSocket-based delivery for both chatroom and private chat.
Demonstrate practical AI integration	Summarization, translation, and search — three well-defined, self-contained AI features.

Objective	Description
Demonstrate sound data lifecycle design	Rooms that expire and clean up their own data; private chats with an opt-in auto-reset policy.
Cover standard messaging conveniences	Delete/unsend, edit, copy, and forward — the small set of per-message actions users expect from a real chat app.
Add lightweight expressive tools	Emoji reactions and a static sticker pack, without the overhead of a full custom-content system.
Single-language backend	An entirely Python backend, simplifying setup, dependencies, and grading/review.

1.4 Intended Audience

- The student/developer building the project
- Instructors or reviewers evaluating the project
- Anyone extending the project later

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification — this document.
JWT	JSON Web Token, used for stateless authentication.
WebSocket	A persistent, bidirectional network connection used for real-time updates.
LLM	Large Language Model — the AI engine behind summarization and translation.
Chatroom	A temporary, multi-user, invite-based chat space (Feature 1).
Private Chat	A persistent one-to-one conversation (Feature 2).
Auto-Reset	A recurring, opt-in mechanism that clears a private chat's messages every 24 hours.
Full-Text Search	A database search technique that matches words or phrases within stored text, used here for in-chat message search.
Unsend	Deleting a previously sent message so it is removed for all participants, not just the sender.

2 Overall Description

2.1 Product Perspective

NexusChat is a standalone web application composed of a frontend client, a Python backend providing REST and WebSocket endpoints, a Supabase-hosted PostgreSQL database with built-in object storage, and the Gemini API for all AI-driven features. There is no dependency on any pre-existing chat product.

2.2 Two Chat Modes, Shared AI Utilities

	Temporary Chatroom	Private Chat
Participants	Multiple (invited members)	Exactly two users
Lifespan	Temporary — ends when admin deletes it or all members go offline	Persistent — stays until a user deletes it, or auto-reset clears it
Deletion control	Room admin, or automatic on all-offline	Either user, manually — or automatic every 24 hrs if both opt in
Summarization	Auto-summary posted as a system message on room expiry, or on-demand at any time	Not applicable
Translation	Available on any message	Available on any message
Search	Search within the room's message history	Search within the chat's message history
Message actions	Sender can delete/unsend, edit (text only), copy, forward	Sender can delete/unsend, edit (text only), copy, forward
Reactions & stickers	Emoji reactions on any message; send static stickers	Emoji reactions on any message; send static stickers
Media sharing	Yes	Yes

2.3 Operating Environment

- Client: modern desktop and mobile web browsers
- Server: a single Python backend service exposing REST and WebSocket endpoints
- Database: Supabase (managed PostgreSQL)
- Storage: Supabase Storage, for shared media files
- AI: Gemini API for summarization and translation

2.4 Design Constraints

- The entire backend shall be implemented in Python — no secondary backend language or runtime.
- The AI summary and translation features depend on the Gemini API; their availability and latency are bound by Google's service and published free-tier rate limits.

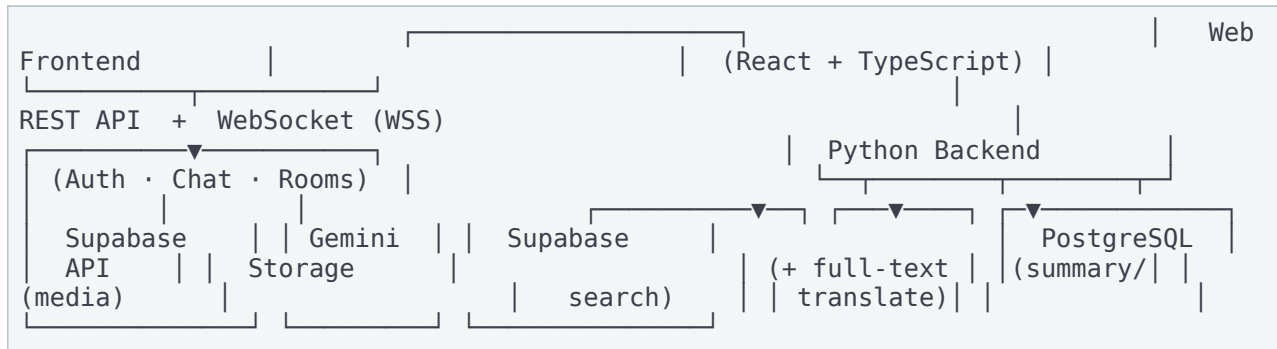
- All client-server communication must occur over HTTPS/WSS.
- No group-admin hierarchy beyond a single "room admin" role is required — this keeps chatroom permissions simple.
- Message search shall use PostgreSQL's built-in full-text search (available natively in Supabase) rather than a separate search engine, to keep the stack minimal.

2.5 Assumptions

- Users have a stable internet connection for real-time delivery to behave as specified.
- The Gemini API free tier provides sufficient quota for development, testing, and demonstration purposes.

3 System Architecture

A simple client-server architecture is sufficient for this scope. The AI service is only invoked for summarization, translation, or when new messages need indexing for search — it is not in the path of ordinary message delivery.



3.1 Technology Stack

The stack below is fixed per the constraints in Section 2.4 — a single Python backend, Supabase for both database and storage, and Gemini as the AI provider.

Layer	Technology
Frontend	React + TypeScript (Vite)
Backend / API	Python — FastAPI
Real-time transport	WebSockets (via FastAPI's native WebSocket support)
Authentication	JWT (Supabase Auth, or a custom JWT layer — either is acceptable)
Database	Supabase (managed PostgreSQL)
Media storage	Supabase Storage
Message search	PostgreSQL full-text search (tsvector/tsquery), native to Supabase
AI provider	Gemini API (Flash model) — summarization and translation

Why Gemini over Groq

Both are free to use for a student project. Gemini is recommended here because summarization and translation are language-quality-sensitive tasks, and Gemini's free tier offers a substantially higher daily request volume alongside strong native multilingual support — a good fit for both AI features this project needs. Groq is optimized primarily for raw inference speed on open-source models, which matters less here than output quality. Either provider can be swapped in with minimal code change, since both expose a simple prompt-in/text-out API.

This remains a small stack overall — no message queue, no caching layer, no microservices, and no separate search engine. A single Python backend and Supabase's built-in capabilities are enough at this scale.

4 Functional Requirements

Requirement IDs (e.g., FR-1.1) allow individual requirements to be referenced from test plans or task tracking.

4.1 Shared — Authentication

Both features require a logged-in user, so a minimal auth layer is needed.

ID	Requirement
FR-1.1	The system shall allow a new user to register with a unique username and email.
FR-1.2	The system shall authenticate users via login with hashed password verification.
FR-1.3	The system shall issue a JWT on successful login and use it to authorize all subsequent requests.
FR-1.4	The system shall allow a user to log out, ending their session.

5 Feature 1 — Temporary Chatroom

5.1 Description

A chatroom is a multi-user space created by one user (the admin), who can invite other users to join. Members exchange text and media in real time. The room is temporary by design: it is destroyed either when the admin deletes it, or automatically when every member has gone offline. Immediately before the room disappears, the system posts an AI-generated summary of the conversation as a special system message, visible to every member who was present.

5.2 Core Behaviors

ID	Requirement
FR-2.1	The system shall allow a user to create a chatroom, becoming its admin.
FR-2.2	The system shall allow the room admin to invite other registered users to the chatroom.
FR-2.3	The system shall allow invited users to join the chatroom.
FR-2.4	The system shall allow room members to send and receive text messages in real time via WebSocket.
FR-2.5	The system shall allow room members to share media files (images, video, audio, documents) within the room.
FR-2.6	The system shall allow the room admin to remove a member from the room.
FR-2.7	The system shall allow the room admin to manually delete the chatroom at any time.
FR-2.8	The system shall automatically mark a chatroom for deletion when all members have gone offline.
FR-2.9	The system shall permanently delete all room messages and media once the room is deleted, whether triggered manually or automatically.

5.3 AI Summary Behavior

A structured summary of what was discussed, delivered as a message inside the chat itself — not a separate report or export.

ID	Requirement
FR-3.1	The system shall allow any room member to request an on-demand AI summary of the chatroom's conversation at any point while the room is active.
FR-3.2	The system shall automatically generate an AI summary of the full conversation immediately before the chatroom is deleted (whether by admin action or all-members-offline expiry).
FR-3.3	The system shall post the AI summary into the chatroom as a distinct system-authored message, sent by "AI" as the message sender.

ID	Requirement
FR-3.4	The AI summary message shall include: the chatroom's name, the list of members present at the time of summarization, and the date and time the summary was generated.
FR-3.5	The AI summary content shall include key discussion points, decisions made, and any action items or deadlines mentioned.
FR-3.6	The system shall deliver the final expiry summary to all members who were present in the room, even after the room itself has been deleted (e.g., via notification or a saved copy outside the room).

Design Note

Because the room's messages are deleted along with the room, the expiry summary is the only surviving record of the conversation. It must be generated and delivered before deletion completes — not after — or the summary generation will have nothing left to read.

6 Feature 2 — Private Chat

6.1 Description

A private chat is a simple one-to-one conversation between two users, similar to a standard WhatsApp chat. Unlike the chatroom, it persists indefinitely by default. A user can delete the conversation manually at any time. Alternatively, both participants can opt into a recurring 24-hour auto-reset: once both have agreed to it, the chat's messages are automatically cleared every 24 hours on a standing basis, without either user needing to trigger it again.

6.2 Core Behaviors

ID	Requirement
FR-4.1	The system shall allow a user to start a private chat with another registered user.
FR-4.2	The system shall allow both participants to send and receive text messages in real time via WebSocket.
FR-4.3	The system shall allow both participants to share media files (images, video, audio, documents) within the chat.
FR-4.4	The system shall persist private chat messages indefinitely unless deleted or reset, per Section 6.3.
FR-4.5	The system shall display each participant's online/offline status and message delivery/read state.

6.3 Deletion and Auto-Reset Behavior

ID	Requirement
FR-5.1	The system shall allow either user to manually delete the private chat at any time, permanently removing its messages and media.
FR-5.2	The system shall allow either user to propose enabling the 24-hour auto-reset for a given private chat.
FR-5.3	The system shall require both participants to accept the auto-reset proposal before it takes effect.
FR-5.4	Once both users have accepted, the system shall automatically clear all messages in that private chat every 24 hours on a recurring basis.
FR-5.5	The system shall allow either user to turn off the auto-reset setting at any time, again requiring both participants' agreement to re-enable it later.
FR-5.6	The system shall notify both participants shortly before each scheduled auto-reset occurs.

Design Note

The auto-reset is a per-chat setting, not a global user preference — it only takes effect for the specific

ID	Requirement
	conversation both users agreed on, and either side can revoke it going forward.

7 Feature 3 — AI Translation

7.1 Description

Available in both the chatroom and private chat, AI translation lets a user translate any individual message into their preferred language on request, without altering the original message for other participants.

ID	Requirement
FR-6.1	The system shall allow a user to select a message in either a chatroom or a private chat and request a translation.
FR-6.2	The system shall allow a user to set a preferred target language, used as the default for translation requests.
FR-6.3	The system shall display the translated text alongside or in place of the original, without modifying the original message for other participants.
FR-6.4	The system shall support translation for both chatroom messages and private chat messages using the same underlying mechanism.
FR-6.5	The system shall handle translation requests through the Gemini API and return the result to the requesting user only.

8 Feature 4 — Message Search

8.1 Description

Available in both the chatroom and private chat, message search lets a user find past messages within a specific conversation by typing a word or a full sentence, without needing to scroll manually through history.

ID	Requirement
FR-7.1	The system shall allow a user to search for messages within a chatroom by word or sentence.
FR-7.2	The system shall allow a user to search for messages within a private chat by word or sentence.
FR-7.3	The system shall return matching messages ranked by relevance, along with the sender and timestamp of each match.
FR-7.4	The system shall restrict search results to conversations the requesting user is a member of.
FR-7.5	The system shall use PostgreSQL full-text search over message content to serve search queries.

Design Note

Search only needs to run within a single chat at a time — there is no cross-chat or global search requirement. This keeps the implementation to a single indexed column and query per chat.

9 Feature 5 — Message Actions

9.1 Description

Available in both the chatroom and private chat, message actions cover the small set of per-message conveniences users expect from a standard chat app: deleting/unsending a sent message, editing a sent text message, copying a message's text, and forwarding a message to another chat. All four actions apply only to messages the requesting user themselves sent, except copy and forward, which may be performed on any message the user can view.

ID	Requirement
FR-8.1	The system shall allow a user to delete (unsend) their own message from a chatroom or private chat, removing it for all participants.
FR-8.2	The system shall allow a user to edit the text content of their own previously sent text message, marking it as edited.
FR-8.3	The system shall not allow editing of media messages (images, video, audio, documents) — edit applies to text messages only.
FR-8.4	The system shall allow a user to copy the text content of any message they can view to their device clipboard.
FR-8.5	The system shall allow a user to forward any message they can view — text or media — to another chatroom or private chat they are a member of.
FR-8.6	The system shall apply delete/unsend and edit actions in real time via WebSocket, so all participants see the change immediately.
FR-8.7	The system shall restrict delete/unsend and edit actions to the message's original sender only.

Design Note

Deleting a message removes its content for everyone in the chat, not just the sender's own view — this keeps the behavior simple and matches how "unsend" typically works in mainstream chat apps. A forwarded message is inserted as a new message in the target chat; it does not link back to or affect the original.

10 Feature 6 — Reactions & Stickers

10.1 Description

Available in both the chatroom and private chat, this feature adds two lightweight, expressive tools on top of ordinary text and media messages: emoji reactions attached to an existing message, and static stickers sent as their own standalone message. The two are distinct — a reaction decorates a message someone already sent, while a sticker is itself a new message, chosen from a fixed sticker pack and posted into the chat like any other message.

10.1.1 Emoji Reactions

Any user in a chat can react to any message with an emoji from a standard set (for example: thumbs up, heart, laugh, surprised, sad). A message can carry multiple reactions from multiple users, and each user's reaction is visible to all participants.

10.1.2 Static Stickers

A sticker is sent as a distinct message type, selected from a fixed, pre-defined sticker pack bundled with the app. Stickers are static images (not animated) and are sent standalone — a sticker message has no accompanying text.

10.2 Requirements

ID	Requirement
FR-9.1	The system shall allow a user to react to any message they can view with an emoji from a standard, predefined set.
FR-9.2	The system shall allow a message to carry multiple reactions from multiple different users simultaneously.
FR-9.3	The system shall allow a user to remove their own reaction from a message.
FR-9.4	The system shall display all reactions on a message, along with which users placed them, to every participant in the chat.
FR-9.5	The system shall allow a user to browse a fixed, predefined sticker pack and send a static sticker as a standalone message.
FR-9.6	The system shall deliver reactions and sticker messages in real time via WebSocket, consistent with ordinary text and media messages.
FR-9.7	The system shall support reactions and stickers identically in both chatroom and private chat contexts.

Design Note

Stickers are static image assets bundled with the app rather than user-uploaded content — there is no sticker creation, custom pack, or animated sticker requirement in this version. This keeps the feature to a fixed asset list plus one new message_type value, rather than a full content-management system.

11 Non-Functional Requirements

Category	Requirement
Performance	Messages shall be delivered in under 1 second under normal network conditions.
Performance	Search queries shall return results in under 2 seconds for typical chat history sizes.
Security	All traffic shall be served over HTTPS/WSS.
Security	Passwords shall be hashed (e.g., bcrypt); plaintext passwords shall never be stored.
Security	All API endpoints shall validate the requester's JWT before granting access.
Security	A user shall only be able to access, search, translate, or summarize chatrooms or private chats they are a member of.
Reliability	WebSocket connections shall reconnect automatically after a brief network interruption.
Reliability	Chatroom and message deletion shall be atomic — a partially deleted room should not be left in an inconsistent state.
Usability	The interface shall clearly distinguish AI-generated summary messages from regular user messages.
Usability	Translated text shall be visually distinguishable from the original message.
Usability	An edited message shall display a visible "edited" indicator to all participants.
Usability	Reactions shall render inline beneath their message without disrupting message order or scroll position.
Reliability	Deleting a message shall remove its content and any attached media from storage, not just hide it from the UI.
Maintainability	The codebase shall be organized into clear modules (auth, chatroom, private chat, AI service, search, message actions, reactions/stickers).
Maintainability	The backend shall be implemented entirely in Python, avoiding mixed-language server components.

12 Database Design

A minimal schema sized to the two chat modes and three AI utilities, hosted entirely on Supabase. Both chatroom and private-chat messages share a single messages table, distinguished by chat type.

12.1 Users

Field	Description
id	Primary key
username	Unique display name
email	Unique email address, used for login
password_hash	Hashed password
preferred_language	Default target language for AI translation
created_at	Account creation timestamp

12.2 Chats

A single table represents both chatrooms and private chats, distinguished by the type field.

Field	Description
id	Primary key
type	"chatroom" or "private"
name	Chatroom name (null for private chats)
admin_id	Foreign key → Users (chatroom admin; null for private chats)
auto_reset_enabled	Boolean — whether the 24-hr auto-reset is active (private chats only)
auto_reset_accepted_by	List of user IDs who have accepted the auto-reset proposal
created_at	Chat creation timestamp
last_reset_at	Timestamp of the last auto-reset (private chats only)

12.3 Chat Members

Field	Description
id	Primary key
chat_id	Foreign key → Chats
user_id	Foreign key → Users

Field	Description
is_online	Boolean — used to detect all-members-offline expiry for chatrooms
joined_at	Timestamp the member joined

12.4 Messages

Field	Description
id	Primary key
chat_id	Foreign key → Chats
sender_id	Foreign key → Users (null when sender is "AI")
sender_type	"user" or "ai" — distinguishes summary messages from regular ones
content	Message text content (null for sticker messages)
message_type	text, image, video, audio, file, or sticker
sticker_id	Foreign key → Stickers (set only when message_type is "sticker")
edited	Boolean flag — true if the message text has been edited (text messages only)
deleted	Boolean flag — true if the message has been deleted/unsent (content cleared)
forwarded_from	Foreign key → Messages (nullable) — the original message this one was forwarded from
search_vector	PostgreSQL tsvector column, generated from content, used for full-text search
created_at	Message send timestamp

12.5 Reactions

Field	Description
id	Primary key
message_id	Foreign key → Messages — the message being reacted to
user_id	Foreign key → Users — who placed the reaction
emoji	The reaction emoji, from the standard predefined set
created_at	Timestamp the reaction was placed

A unique constraint on (message_id, user_id, emoji) prevents a user from placing the identical reaction on the same message twice; removing a reaction deletes the corresponding row.

12.6 Stickers

Field	Description
id	Primary key
pack_name	Name of the sticker pack this sticker belongs to
image_url	Storage location of the static sticker image
display_order	Sort order within its pack, for consistent picker display

The Stickers table is a fixed, app-bundled reference list (seeded at setup) rather than user-generated content — no upload or creation flow is required for it.

12.7 Attachments

Field	Description
id	Primary key
message_id	Foreign key → Messages
file_url	Supabase Storage location of the file
file_name	Original file name
file_size	Size in bytes
mime_type	MIME type of the file

12.8 Room Summaries

Stores the final expiry summary separately from the (now-deleted) chatroom, so it can still be delivered to former members afterward.

Field	Description
id	Primary key
chat_id	The chatroom's ID at time of summarization (chat itself may since be deleted)
chat_name	The chatroom's name, preserved for the summary header
members_present	List of usernames present at the time of summarization
summary_text	The AI-generated summary content
generated_at	Timestamp the summary was generated
trigger	"manual" or "expiry" — how the summary was triggered

Search Implementation Note

The search_vector column on Messages is a generated PostgreSQL tsvector column with a GIN index. A search

Field	Description
	query converts the user's word or sentence into a tsquery and matches it against search_vector, scoped to a single chat_id per FR-7.4. This is entirely native to Supabase's PostgreSQL — no external search service is required.

13 API Overview

13.1 Authentication

```
POST    /register POST    /login POST    /logout
```

13.2 Chatroom

```
POST    /chatrooms                # create a room POST
/chatrooms/{id}/invite            # invite a user POST
# accept invite / join DELETE /chatrooms/{id}/members/{userId} # admin removes a
member DELETE /chatrooms/{id}    # admin deletes room POST
/chatrooms/{id}/summarize        # request an on-demand summary GET
/chatrooms/{id}/messages        # fetch message history GET
/chatrooms/{id}/search?q=...    # search messages in the room
```

13.3 Private Chat

```
POST    /private-chats            # start a private chat GET
/private-chats/{id}/messages      # fetch message history GET
/private-chats/{id}/search?q=...  # search messages in the chat DELETE
/private-chats/{id}              # manually delete the chat POST
/private-chats/{id}/auto-reset/propose # propose enabling auto-reset POST
/private-chats/{id}/auto-reset/accept # accept the proposal POST
/private-chats/{id}/auto-reset/disable # turn off auto-reset
```

13.4 Messages & Media (shared)

```
POST    /messages PUT    /messages/{id}                # edit a text message DELETE
/messages/{id}            # delete / unsend a message POST
/messages/{id}/forward    # forward a message to another chat POST
/messages/{id}/reactions  # add a reaction to a message DELETE
/messages/{id}/reactions/{emoji} # remove own reaction POST
/media/{id}              /upload GET
```

13.5 Stickers (shared)

```
GET    /stickers                # list available sticker packs and stickers POST
/messages/sticker              # send a sticker as a message
```

13.6 AI (shared)

```
POST    /ai/translate    # translate a single message POST
# on-demand chatroom summary (see 13.2) /ai/summarize
```

14 WebSocket Events

14.1 Client → Server

Event	Description
join_room	Client joins a specific chat (chatroom or private chat) to receive its events.
leave_room	Client leaves a chat.
send_message	Client sends a new message.
edit_message	Client edits the text of their own previously sent message.
delete_message	Client deletes/unsends their own message.
forward_message	Client forwards a message to another chat.
send_sticker	Client sends a sticker as a standalone message.
add_reaction	Client reacts to a message with an emoji.
remove_reaction	Client removes their own reaction from a message.
typing / stop_typing	Client signals typing status.

14.2 Server → Client

Event	Description
receive_message	Server pushes a new message (text, media, or sticker) to chat participants.
message_edited	Server broadcasts that a message was edited, with the updated text.
message_deleted	Server broadcasts that a message was deleted/unsent, so clients remove or gray it out.
reaction_added	Server broadcasts that a reaction was placed on a message.
reaction_removed	Server broadcasts that a reaction was removed from a message.
user_online / user_offline	Server broadcasts presence changes (used to detect chatroom all-offline expiry).
room_expired	Server notifies members that a chatroom has been deleted, along with its summary.
chat_reset	Server notifies both private-chat participants that an auto-reset has occurred.
ai_summary_ready	Server delivers a newly generated AI summary message.
message_translated	Server returns a translated message to the requesting client.

15 User Interface Pages

15.1 Authentication

- Login
- Register

15.2 Application

- Dashboard — list of active chatrooms and private chats
- Chatroom View — messaging, media sharing, sticker picker, emoji reactions, invite members, in-chat search bar, per-message menu (translate, copy, forward, react, and — for own messages — edit/delete), summarize button (admin sees delete-room option)
- Private Chat View — messaging, media sharing, sticker picker, emoji reactions, in-chat search bar, per-message menu (translate, copy, forward, react, and — for own messages — edit/delete), delete-chat option, auto-reset settings toggle
- Profile / Settings — includes preferred language selection for translation

16 Out of Scope

To keep this achievable as a student project, the following are explicitly not included in this version:

- Friend/contact system, friend requests, blocking
- Group admin hierarchies beyond a single room-admin role
- Message pinning or reply-to threading
- Editing of media messages (edit is text-only, per Section 9.1)
- Animated stickers, custom sticker uploads, or user-created sticker packs — only a fixed, static, app-bundled pack
- AI features beyond summarization, translation, and search (no Ask-AI or smart replies)
- Voice or video calling
- Push notifications outside the app
- Cross-chat or global search — search is scoped to one chat at a time

Any of these can be added later as extensions once the core build is complete and stable.

17 Success Criteria

The project will be considered complete when it:

- Supports secure registration, login, and logout.
- Allows creating a chatroom, inviting members, and real-time messaging with media sharing.
- Correctly deletes a chatroom when the admin removes it, or when all members go offline.
- Generates and delivers an AI summary — both on-demand and automatically at room expiry — with chatroom name, members present, and timestamp.
- Supports a persistent one-to-one private chat with real-time messaging and media sharing.
- Allows manual deletion of a private chat by either participant.
- Supports a mutually-agreed, recurring 24-hour auto-reset for a private chat, toggleable by either user.
- Allows translating any message, in either chat mode, into a user's preferred language.
- Allows searching by word or sentence within a chatroom or private chat's message history.
- Allows a sender to delete/unsend or edit (text only) their own messages, with changes reflected in real time for all participants.
- Allows any viewable message to be copied or forwarded to another chat.
- Allows any user to react to a message with an emoji, and to send static stickers from a fixed pack as standalone messages.
- Runs on an all-Python backend with Supabase as the sole database and storage provider.
- Is deployed and accessible for demonstration or evaluation.

Document Note

This is a deliberately narrow SRS. Resist the urge to add features mid-build — the current scope is what makes this finishable. Anything from the Out of Scope list is a good "future work" talking point for a project writeup, without needing to actually build it.